QUANTUM CHESS

Quantum Game Project — CS5331/4331

Introduction to Quantum Computing | Texas Tech University

1. Project Overview

Quantum Chess is a chess variant where pieces can exist in superposition across two squares simultaneously. A piece is not observed — and therefore not collapsed to a single location — until it is involved in a capture attempt or a dedicated measurement move. Entangled pieces affect each other's collapse in a way that mirrors a real Bell state.

The project directly demonstrates three core quantum concepts from the course:

- Superposition — pieces occupy two squares at once (Hadamard gate)
- Measurement / Collapse — capturing a superposed piece triggers a real quantum measurement
- Entanglement — two pieces linked via a CNOT gate always collapse together

> **Novelty Statement**
> Unlike Google's Quantum Chess, which uses a full classical chess engine with quantum-flavored rules, our version performs real quantum gate operations (H, CNOT, Measure) on IBM Quantum hardware for key game events. The collapse mechanic is a genuine quantum measurement — not a simulated one — making this a pedagogical quantum computing demo embedded in a game.

2. Technology Stack

Component	Tool / Library	Purpose
Quantum Backend	Qiskit + IBM Quantum Runtime	Real H, CNOT, Measure gates
Quantum Simulator	Qiskit Aer	Fast local simulation for dev/demo fallback
Game Logic	Python (custom)	Chess rules + quantum rule extensions
UI / Rendering	Pygame or HTML/JS	Board, ghost pieces, collapse animations
Circuit Visuals	IBM Quantum Composer	Slide diagrams for presentation

3. Project Structure

```
quantum_chess/
├── quantum_engine.py      # Qubit state management, H gate, CNOT, measurement
├── entanglement.py        # Bell state creation, correlated collapse logic
├── board.py               # Classical board state, piece positions, move rules
├── quantum_rules.py       # Superposition move, entangle move, measure move
├── game_manager.py        # Turn flow, win condition, event orchestration
├── renderer.py            # Board drawing, ghost pieces, collapse animations
├── ui_components.py       # State panel, move history, quantum event log
└── main.py                # Entry point, game loop
```

4. Team Division (3 Members)

Person A — Quantum Backend

Owns all qubit state logic. Everything involving real quantum gates lives in this module.

- Represent each piece in superposition as a qubit (PennyLane or Qiskit)
- Implement Hadamard gate to put a piece into superposition
- Implement CNOT-based entanglement between two pieces
- Send real measurement circuits to IBM Quantum hardware for collapse resolution
- Write unit tests: does superposition work? Does measurement collapse correctly?
- Own the hardware / simulator toggle for demo reliability

Person B — Game Logic & Rules

Owns classical chess rules plus all quantum rule extensions. Connects to Person A's quantum engine to resolve quantum events.

- Implement standard chess board, piece movement, and turn management
- Define the three new quantum moves:
  - Superposition move — split a piece across two squares
  - Measure move — force collapse of a piece or the full board
  - Entangle move — link two pieces so their collapses are correlated
- Handle capture attempts against superposed pieces (trigger measurement)
- Handle win/loss conditions under quantum uncertainty

Person C — UI & Visualization

Owns everything the player sees and interacts with. Connects to Person B's game manager to reflect live board state.

- Build the chess board UI (Pygame or simple web frontend)
- Visually represent superposition — semi-transparent ghost pieces on both squares
- Animate collapse — ghost pieces snap to one square with a visual effect
- Show entanglement links between pieces (glowing line between entangled pieces)
- Display quantum state panel: which pieces are superposed, entanglement map

5. Shared Data Contract (agree on this first — Week 1)

Before anyone writes a line of code, all three team members must agree on how a piece is represented. This shared structure is the contract that lets all three workstreams proceed in parallel without blocking each other.

```
# Shared piece representation — agree on this in Week 1
piece = {
    "type":          "knight",
    "color":         "white",
    "positions":     ["e4"],          # classical: one position
    # OR
    "positions":     ["e4", "g5"],    # superposition: two positions
    "superposed":    True,
    "qubit_id":      3,               # index into the quantum register
    "entangled_with": [7],            # qubit IDs of entangled pieces
}
```

6. IBM Quantum Integration

Setup

- Install: pip install qiskit qiskit-ibm-runtime qiskit-aer
- Create a free account at quantum.ibm.com
- Save your API token once:

```
from qiskit_ibm_runtime import QiskitRuntimeService
QiskitRuntimeService.save_account(channel="ibm_quantum", token="YOUR_TOKEN")
```

Hardware vs. Simulator Strategy

Running every move on real IBM hardware is impractical due to queue times. The recommended hybrid approach:

Game Event	Backend	Reason
Regular moves	Classical (instant)	No quantum needed
Superposition move	Qiskit Aer	Fast, no queue wait
Capture of superposed piece	IBM Real Hardware	Showcase moment — genuine quantum
Entanglement collapse	IBM Real Hardware	Demonstrates Bell state
Presentation fallback	Qiskit Aer	Reliable for live demo

Hardware Toggle Pattern

```
USE_REAL_HARDWARE = False  # Flip to True for showcase moments


def measure_qubit(circuit):
```

```
    if USE_REAL_HARDWARE:
        service = QiskitRuntimeService(channel='ibm_quantum')
        backend = service.least_busy(operational=True, simulator=False)
    else:
        from qiskit_aer import AerSimulator
        backend = AerSimulator()
    sampler = Sampler(backend)
    job = sampler.run([circuit], shots=1)
    return job.result()
```

7. Development Timeline

Week	Milestone
1	All three: agree on shared data contract (piece representation). Do NOT write game code yet.
2	A: single-qubit superposition + measurement working. B: classical chess board + legal moves. C: static board renders correctly.
3	Integration sprint — A+B connect quantum events to game logic. C pulls live board state from B.
4	All quantum rules working (superposition, capture collapse, entanglement). Ghost pieces and collapse animations in UI.
5	Bug fixing, edge cases, IBM Quantum integration tested. Presentation prepared and rehearsed.

8. Quantum Circuits to Implement

Superposition Move (Hadamard gate)

```
qc = QuantumCircuit(1, 1)
qc.h(0)          # |0⟩ → |+⟩ = (|0⟩ + |1⟩) / √2
qc.measure(0, 0) # Collapses to |0⟩ or |1⟩ with equal probability
```

Entanglement (Bell State via CNOT)

```
qc = QuantumCircuit(2, 2)
qc.h(0)           # Piece A in superposition
qc.cx(0, 1)       # Entangle piece B with piece A
qc.measure([0,1],[0,1])  # Always collapses to |00⟩ or |11⟩ — never |01⟩ or |10⟩
```

> **Presentation Tip**
> Show your audience that entangled pieces ALWAYS collapse together (00 or 11, never 01 or 10). This is the Bell state in action — not a simulation, but a real physical property of entangled qubits running on IBM hardware. That is your strongest demo moment.

9. Installation & Running

Dependencies

```
pip install qiskit qiskit-ibm-runtime qiskit-aer pygame
```

Run the game

```
python main.py
```

Run module tests individually

```
python quantum_engine.py    # Tests superposition + measurement
python entanglement.py      # Tests Bell state creation
python board.py             # Tests classical chess rules
```

10. References & Resources

- IBM Quantum Composer: quantum.ibm.com/composer/files/new
- Qiskit Documentation: docs.quantum.ibm.com
- PennyLane (alternative quantum backend): pennylane.ai
- Quantum Chess (Google / SplitInfinity) — original inspiration: store.steampowered.com/app/2802710
- Nielsen & Chuang — Quantum Computation and Quantum Information, Chapter 1–2
- Course material: CS5331/4331, TTU